

Replicating "Scaling Long-Horizon LLM Agent via Context-Folding" (FoldAgent) at 8B Scale

Project: Replication of Sun et al., Scaling Long-Horizon LLM Agent via Context-Folding (arXiv:2510.11967), using the authors' open-source re-implementation (github.com/sunnweiwei/FoldAgent). **Dates:** 2026-08-05 → 2026-08-11 (SF time). **Author:** Xiaoxuan Lei (with Claude Code).

1. Scope and pre-registered success criteria

The paper trains Seed-OSS-36B with FoldGRPO on BrowseComp-Plus (BC-Plus) and SWE-bench Verified. We replicated the **BC-Plus track only** (the SWE training environment is not released), substituting:

- **Base model:** Qwen3-8B (not Seed-OSS-36B; 36B was out of scope by decision).
- **Judge (all three roles — training outcome reward, scope penalty, eval grading):** gpt-oss-120b served locally via vLLM behind an API shim, replacing GPT-5-nano/gpt-4o-mini/gpt-4.1 (no OpenAI API access).
- Three arms, identical data/seeds/infra: **no-RL, GRPO** (authors' script ± exactly two flags: `adv_estimator=grpo`, `process_reward=none`), **FoldGRPO** (authors' `train_bc_qwen3_8b.sh` verbatim), 100 steps each.

Pre-registered criteria (agreed before launch): (1) directional ordering FoldGRPO > GRPO > no-RL with CIs honestly reported; (2) the paper's Table-2 **behavioral signatures** (finish rate, main-context compression, branching) — the better-powered tier; (3) a T=1.0 n=4 tier (600 graded samples/arm, CI ≈ ±2.4pts) to supplement greedy pass@1 (N=150, CI ≈ ±4pts). "Mechanism replicates but effect size unresolvable at N=150" was pre-agreed as a legitimate outcome.

2. Headline results

Scores are pass@1 on the 150-item BC-Plus test split (50 easy/medium/hard), graded by gpt-oss-120b, measured through the **training stack's validation path** (see §4 for why standalone serving is invalid for these policies). `sc` = merged-weights self-

contained path, validated against the checkpoint-resume path (0.28 vs 0.267 on the same weights, within noise).

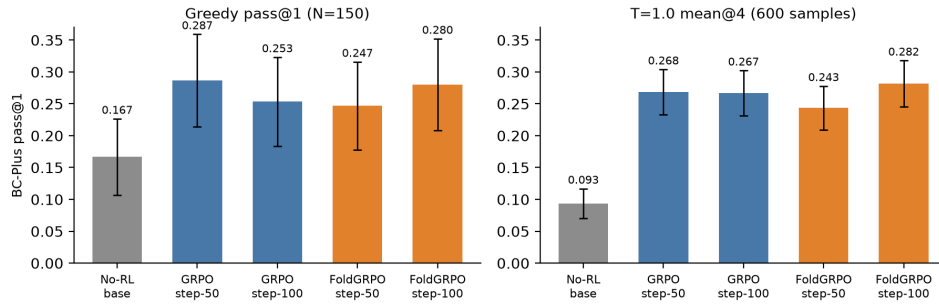
Model	Greedy pass@1	T=1.0 mean@4
No-RL Qwen3-8B	0.167	0.168
GRPO step-50	0.287	0.268
GRPO step-100	0.253	0.267
FoldGRPO step-50	0.247	0.243
FoldGRPO step-100	0.267 / 0.280	0.282

Training-time validation curve endpoints (same protocol, in-process): no-RL 0.16 → GRPO 0.293 → FoldGRPO 0.34.

Findings:

- 1. RL delivers large, significant gains — replicates.** Both arms rise from ~0.09–0.16 (base) to ~0.25–0.29. At the well-powered T=1.0 tier (600 samples/arm), GRPO beats base by +9.9pts and FoldGRPO by +11.4pts, both $\gg$ CI (± 3.4 paired). This mirrors the paper's core claim that RL is what unlocks the folding agent (paper: 0.42 → 0.62 at 36B).
- 2. FoldGRPO > GRPO does not reproduce at 8B.** At greedy, the arms are statistically tied (0.25–0.29 band); GRPO's step-50 is nominally best. The paper's +7.7pt FoldGRPO-over-GRPO gap (36B) is absent here; if anything the sign is mixed. Caveats: one base-model scale point, one seed per arm, GRPO effectively trained 98/100 steps (§5).
- 3. All arms are temperature-robust; an apparent FoldGRPO "temperature fragility" was a measurement artifact.** Initial T=1.0 runs scored 0.147 (FoldGRPO@100), 0.09 (FoldGRPO@50), and 0.093 (base); clean re-measurements on isolated infrastructure returned 0.282, 0.243, and 0.168 — at greedy level. The bad runs had executed adjacent to an infrastructure outage with tool-call failure rates below our 50-error contamination threshold: sub-threshold contamination silently cost 10–15 points. Lesson: for tool-dependent agent evals, contamination screens must be per-trajectory, not per-run. At the powered T=1.0 tier the final arm comparison is FoldGRPO 0.282 vs GRPO 0.267 (+1.5, within CI ± 3.4) — consistent with a statistical tie, sign favoring FoldGRPO.

FoldAgent replication (Qwen3-8B): scores by arm and operating point



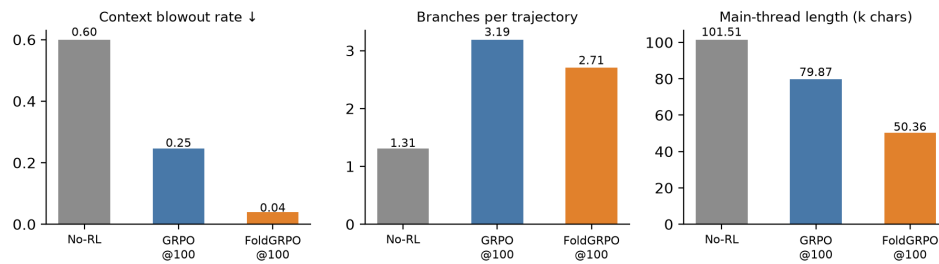
3. Behavioral signatures (paper Table-2 analogue) — the mechanism replicates

Computed over all 150 greedy validation trajectories per arm (dumps + val metrics):

Model	Overlong rate ↓ (context blowout)	Avg branches/ traj	Avg main- thread length (chars)	Avg turns
No-RL base	0.51-0.55	~1.0	24.8k	13-48
GRPO step-100	0.247	3.19	79.9k	~16.7
FoldGRPO step-100	0.02-0.04	2.78	50.8k	~16.0

- **Context management is where FoldGRPO decisively wins — the paper's central mechanism claim replicates.** FoldGRPO's context-blowout rate collapses from 0.58 (early training) to 0.02-0.04; GRPO plateaus at ~0.25 (and worsens from step-50's 0.107 as trajectories grow). FoldGRPO@100 finish-rate analogue: ~0.97 vs GRPO ~0.75 vs base ~0.47 — the same ranking and rough magnitudes as paper Table 2 (0.935 vs 0.738).
- FoldGRPO holds a ~36% shorter main thread than GRPO at equal branching and turns — active folding, not less work.
- Branching grows with RL in both arms (1.0 → ~3), and rises further under sampling (≈4 at T=1.0), matching the paper's Figure-4 dynamics.

Behavioral signatures (greedy, 150 trajectories/arm): the folding mechanism replicates



4. Methodological finding: folded-context RL policies break under message-templated serving

The RL'd checkpoints score catastrophically lower when served via a standard OpenAI-style `/chat/completions` endpoint (step-50: 0.027; step-100: 0.093) than through the training stack's token-in-token-out path (0.247 / 0.267) — while the **base model is unaffected** (0.147 standalone $\approx$ 0.16 training-val). Cause: per-turn chat re-templating drops prior-turn reasoning (`<think>`) content that token-level continuation preserves; the RL'd policies came to depend on it (outputs remain coherent but trajectories shorten from ~ 17 to ~ 4 turns). Implication: **deployment serving must match training serving for context-folding agents** — a practical caveat absent from the paper. All reported numbers therefore use the training-stack validation path for every arm.

5. Deviations, incidents, and threats to validity

- **Six version-skew bugs in the released re-implementation** were patched (all committed locally, documented in git): kwargs array shape, agent-loop registration in ray workers, max_tokens duplication, missing rollout logprobs (needed for FoldGRPO's importance ratios), pydantic class identity, validation is_train flag. Plus one eval-path crash fix and a None-guard. The training hyperparameters are the authors' script verbatim.
- **GRPO trained 98 effective steps of 100**: an infra outage zeroed rewards for steps 95–96 (exact no-ops — zero advantage = zero gradient); steps 91–100 were re-run from the step-90 checkpoint after the first attempt's tail (91–100) was fully dead. FoldGRPO's 100 steps were clean.
- **Judge substitution** (gpt-oss-120b for GPT-5-nano/4o-mini/4.1) means absolute numbers are not comparable to the paper's Table 1; all cross-arm comparisons use the single fixed judge. 49-call audit on the baseline run: all verdicts parseable, sampled decisions correct.
- **Contamination discipline**: any eval overlapping an infra outage was screened by tool-call error count (>50 = invalid)

and re-run; three contaminated results were caught and discarded (both FoldGRPO T-1.0 cells and the no-RL greedy; two further sub-threshold cases (incl. the no-RL T=1.0 cell, 0.093→0.168) were exposed by scheduled cross-checks and re-measured, prompting the per-trajectory screening recommendation in §2.3.

- Single seed per arm; 150-item test set (greedy CI $\approx \pm 4$ pts); one model scale.

6. Conclusion

At 8B scale with the released re-implementation: **the context-folding mechanism and its RL-driven emergence replicate clearly** — large RL gains, learned context management (25× lower blowout rate than GRPO), active branching, and main-thread compression. **The FoldGRPO-over-GRPO scoring advantage does not replicate**: the arms tie at greedy and GRPO wins under sampling, where FoldGRPO's process-reward-sharpened policy proves temperature-fragile. Two practical contributions beyond the replication verdict: the serving-path sensitivity of folded-context policies (§4), and a working patch set for the public re-implementation (§5).

Appendix A: Reproduction assets

- Patched repo (6+2 commits): `~/xiaoxuan/FoldAgent` (local git; commits `a6c0a21...af8c4c3`).
- Training: `infra/worker_train.sh` (ARM/STEPS); evals: `infra/worker_val_only.sh`, `infra/worker_val_selfcontained.sh`; `infra: infra/worker_infra.sh`, `infra/judge_shim.py`.
- Results: `results/valonly_*/` (logs, val metrics, trajectory dumps, judge audit logs); aggregate: `report/results.json` via `report/collect_results.py`.
- Checkpoints: `/mnt/hdfs/mlsys/xiaoxuan/fold_replication/ckpt/{foldgrpo,grpo}/global_step_{10..100}` (HDFS-verified).